

22.06.2026

Sprawozdanie z Podstaw Baz Danych

Temat: NoSQL

Grupa: 6, Zestaw A

Konrad Wydmuch

1. Podać nazwę tematu o numerze 5

```
FOR t IN tematy
FILTER t._key == "5"
RETURN { "temat": t.TEMAT }
```

Query 1 elements 0.304 ms JSON Table x

```
1 [
2 {
3   "temat": "BAZA DANYCH - SPOLEM"
4 }
5 ]
```

Download CSV Copy To Editor

2. Podać płeć i nazwiska pracowników zaczynające się od litery K

```
FOR p IN pracownicy
FILTER LIKE(p.NAZWISKO, "K%")
RETURN { "plec": p.PLEC, "nazwisko": p.NAZWISKO }
```

Query 6 elements 0.320 ms JSON Table x

```
1 [
2 {
3   "plec": "M",
4   "nazwisko": "KOTULA"
5 },
6 {
7   "plec": "M",
8   "nazwisko": "KULANEK"
9 },
10 {
11   "plec": "M",
12   "nazwisko": "KULAWIK"
13 },
14 {
15   "plec": "M"
```

Download CSV Copy To Editor

3. Podać nazwy tematów, których kierownikiem jest pracownik MISIURA

FOR p IN pracownicy

FILTER LIKE(p.NAZWISKO, "MISIURA")

FOR t IN tematy

FILTER t.NR_PRAC_KT == p._key

RETURN { "temat": t.TEMAT }

The screenshot shows a query result in a database interface. The top bar indicates 'Query' mode, '4 elements', and a execution time of '0.459 ms'. The result is displayed in JSON format, showing four objects, each with a 'temat' key. The values are 'ADA EDYTOR', 'PRZETWARZANIE ROZPROSZONE', 'GENERATOR DZWIEKU', and 'KARTA MONITORA GRAFICZNEGO'. The interface includes a 'Download' button, a 'CSV' button, and a 'Copy To Editor' button.

```
1 [
2 {
3   "temat": "ADA EDYTOR"
4 },
5 {
6   "temat": "PRZETWARZANIE ROZPROSZONE"
7 },
8 {
9   "temat": "GENERATOR DZWIEKU"
10 },
11 {
12   "temat": "KARTA MONITORA GRAFICZNEGO"
13 }
14 ]
```

4. Podaj nazwiska pracowników i daty wypłat, które otrzymały kobiety urodzone przed 1970 rokiem według daty wypłaty (użyj DATE_YEAR).

FOR p IN pracownicy

FILTER p.PLEC == "K"

FILTER DATE_YEAR(p.DATA_UR) < 1970

FOR w in wypłaty

FILTER w.NR_PRAC == p._key

SORT w.DATA_WYPL

COLLECT nazwisko = p.NAZWISKO INTO daty = w.DATA_WYPL

RETURN { "nazwisko": nazwisko, "Data": daty }

The screenshot shows a query result in a database interface. The top bar indicates 'Query' mode, '15 elements', and a execution time of '3.696 ms'. The result is displayed in JSON format, showing two objects. Each object has a 'nazwisko' key and a 'Data' key. The 'Data' key contains an array of dates. The first object has 'nazwisko': 'BIERNAT' and a list of five dates. The second object has 'nazwisko': 'FULAK' and a list of one date. The interface includes a 'Download' button, a 'CSV' button, and a 'Copy To Editor' button.

```
1 [
2 {
3   "nazwisko": "BIERNAT",
4   "Data": [
5     "1983-04-06 00:00:00",
6     "1984-04-25 00:00:00",
7     "1985-05-24 00:00:00",
8     "1987-07-05 00:00:00",
9     "1987-07-13 00:00:00",
10    "1990-09-17 00:00:00"
11   ]
12 },
13 {
14   "nazwisko": "FULAK",
15   "Data": [

```

5. Podać w odwrotnej kolejności alfabetycznej nazwiska pracowników mężczyzn, którzy mają wypłaty w temacie DYSK lub ADA EDYTOR, wraz z nazwą tematu. Wartości nie powinny się powtarzać (8).

```
FOR p IN pracownicy
```

```
  FILTER p.PLEC == "M"
```

```
    FOR w in wypłaty
```

```
      FILTER w.NR_PRAC == p._key
```

```
        FOR t IN tematy
```

```
          FILTER t._key == w.NR_TEM
```

```
          FILTER t.TEMAT IN ["DYSK", "ADA EDYTOR"]
```

```
          COLLECT nazwisko = p.NAZWISKO, temat = t.TEMAT
```

```
        SORT nazwisko DESC
```

```
      RETURN { "nazwisko": nazwisko, "temat": temat }
```

Query 8 elements 5.953 ms

JSON Table

```
1 [
2   {
3     "nazwisko": "WIREK",
4     "temat": "DYSK"
5   },
6   {
7     "nazwisko": "TADEECZEK",
8     "temat": "ADA EDYTOR"
9   },
10  {
11    "nazwisko": "SZCZFRBTEC"
```

Download CSV Copy To Editor

6. Podać wszystkie funkcje pełnione przez pracowników w tematach, które rozpoczęły się i zostały odebrane przed rokiem 1985 (10). Każda funkcja powinna pojawić się tylko raz

```
FOR t IN tematy
```

```
  FILTER DATE_YEAR(t.DATA_ROZP) < 1985 AND DATE_YEAR(t.DATA_ODB) < 1985
```

```
    FOR p in przydziały
```

```
      FILTER p.NR_TEM == t._key
```

```
      COLLECT funkcja = p.KOD_FUNKCJI
```

```
    RETURN {"Funkcja": funkcja}
```

Query 5 elements 3.529 ms

JSON Table

```
1 [
2   {
3     "Funkcja": "ANL"
4   },
5   {
6     "Funkcja": "KRW"
7   },
8   {
9     "Funkcja": "PRG"
10  },
11  {
```

Download CSV Copy To Editor

7. Podać nazwy tematów i nazwy ich tematów nadrzędnych (9).

FOR t in tematy

FOR tn in tematy

FILTER tn._key == t.NR_TEM_NADRZ

RETURN {"Temat": t.TEMAT, "Temat nadrzedny": tn.TEMAT}

Query 9 elements 0.669 ms JSON Table

```
1 [
2   {
3     "Temat": "ADA EDYTOR",
4     "Temat nadrzedny": "ADA SRODOWISKO"
5   },
6   {
7     "Temat": "ADA KOMPILATOR",
8     "Temat nadrzedny": "ADA SRODOWISKO"
9   },
10  {
11    "Temat": "KARTA PROCESORA"
```

Download CSV Copy To Editor

7A. A jak to zrobić, aby zgrupować je pod nadrzędnym?

FOR t in tematy

FOR tn in tematy

FILTER tn._key == t.NR_TEM_NADRZ

COLLECT tematN = tn.TEMAT INTO tematP = t.TEMAT

RETURN {"Temat": tematN, "Temat nadrzedny": tematP}

Query 2 elements 0.558 ms JSON Table

```
1 [
2   {
3     "Temat": "ADA SRODOWISKO",
4     "Temat nadrzedny": [
5       "ADA EDYTOR",
6       "ADA KOMPILATOR"
7     ]
8   },
9   {
10    "Temat": "ZESTAW KOMPUTEROWY",
11    "Temat nadrzedny": [
```

Download CSV Copy To Editor

8a. Podać ile przydziałów ma pracownik MISIURA.

```
FOR p IN pracownicy
```

```
  FILTER p.NAZWISKO == "MISIURA"
```

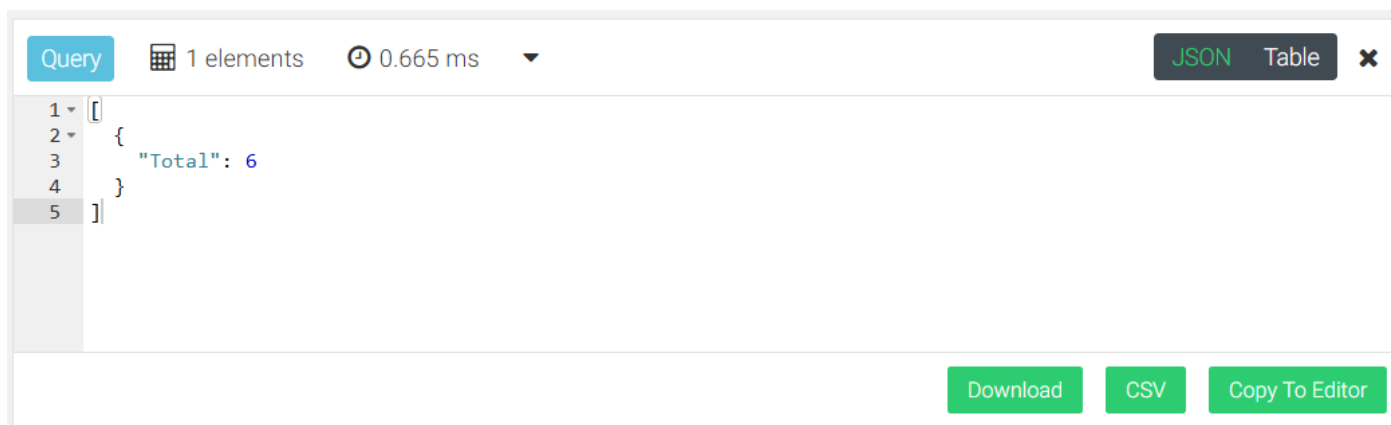
```
    FOR prz in przydzialy
```

```
      FILTER prz.NR_PRAC == p._key
```

```
      COLLECT AGGREGATE
```

```
        total = COUNT( p._key)
```

```
  RETURN {"Total": total}
```



The screenshot shows a query result interface. At the top, there's a header bar with a 'Query' tab, a table icon, '1 elements', a clock icon, '0.665 ms', and a dropdown arrow. On the right, there are tabs for 'JSON' (selected) and 'Table', and a close button 'x'. The main area displays a JSON array with one element:

```
[{"Total": 6}]
```

. The array is numbered 1 to 5 on the left. At the bottom right, there are three green buttons: 'Download', 'CSV', and 'Copy To Editor'.

8b. Podać ile przydziałów ma pracownik MISIURA do poszczególnych funkcji.

```
FOR p IN pracownicy
```

```
  FILTER p.NAZWISKO == "MISIURA"
```

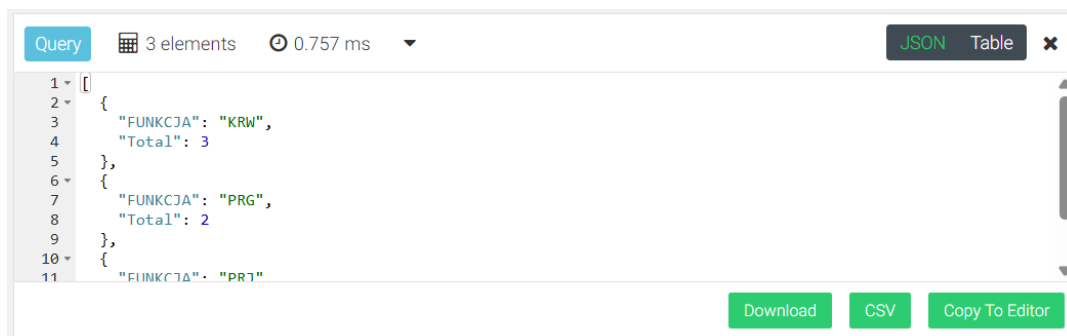
```
    FOR prz in przydzialy
```

```
      FILTER prz.NR_PRAC == p._key
```

```
      COLLECT funkcja = prz.KOD_FUNKCJI
```

```
      WITH COUNT INTO total
```

```
  RETURN {"FUNKCJA": funkcja , "Total": total}
```



The screenshot shows a query result interface. At the top, there's a header bar with a 'Query' tab, a table icon, '3 elements', a clock icon, '0.757 ms', and a dropdown arrow. On the right, there are tabs for 'JSON' (selected) and 'Table', and a close button 'x'. The main area displays a JSON array with three elements:

```
[{"FUNKCJA": "KRW", "Total": 3}, {"FUNKCJA": "PRG", "Total": 2}, {"FUNKCJA": "PRJ", "Total": 1}]
```

. The array is numbered 1 to 11 on the left. At the bottom right, there are three green buttons: 'Download', 'CSV', and 'Copy To Editor'.

9. Podać sumę wszystkich wypłat w temacie DYSK

```
FOR t IN tematy
  FILTER t.TEMAT == "DYSK"
  FOR w IN wypłaty
    FILTER w.NR_TEM == t._key
  COLLECT AGGREGATE
    total = SUM(w.KWOTA)
RETURN {"Suma wypłat": total }
```

The screenshot shows a query result interface. At the top, there's a header bar with 'Query' in a blue button, '1 elements' with a grid icon, and a timer '0.792 ms'. On the right, there are tabs for 'JSON' (highlighted in green) and 'Table', along with a close button 'x'. The main area displays a JSON array with one element:

```
1 [
2   {
3     "Suma wypłat": 5049
4   }
5 ]
```

 At the bottom, there are three green buttons: 'Download', 'CSV', and 'Copy To Editor'.

10. Podać średnią wypłatę wszystkich pracowników będących kierownikami tematów

```
FOR t IN tematy
  FOR w IN wypłaty
    FILTER w.NR_PRAC == t.NR_PRAC_KT
  COLLECT pracownik = t.NR_PRAC_KT
    AGGREGATE srednia = AVERAGE(w.KWOTA)
RETURN {"Pracownik": pracownik, "Srednia wypłat": srednia }
```

The screenshot shows a query result interface. At the top, there's a header bar with 'Query' in a blue button, '16 elements' with a grid icon, and a timer '11.202 ms'. On the right, there are tabs for 'JSON' (highlighted in green) and 'Table', along with a close button 'x'. The main area displays a JSON array with three elements, each containing an employee ID and their average payment:

```
1 [
2   {
3     "Pracownik": "1",
4     "Srednia wypłat": 537.777777777778
5   },
6   {
7     "Pracownik": "11",
8     "Srednia wypłat": 510
9   },
10  {
11   "Pracownik": "18"
12 }
```

 At the bottom, there are three green buttons: 'Download', 'CSV', and 'Copy To Editor'.

11. Podać ilość i sumę wypłat w poszczególnych tematach (wyświetl nazwę tematu w zestawieniu)

FOR t IN tematy

FOR w IN wypłaty

FILTER w.NR_TEM == t._key

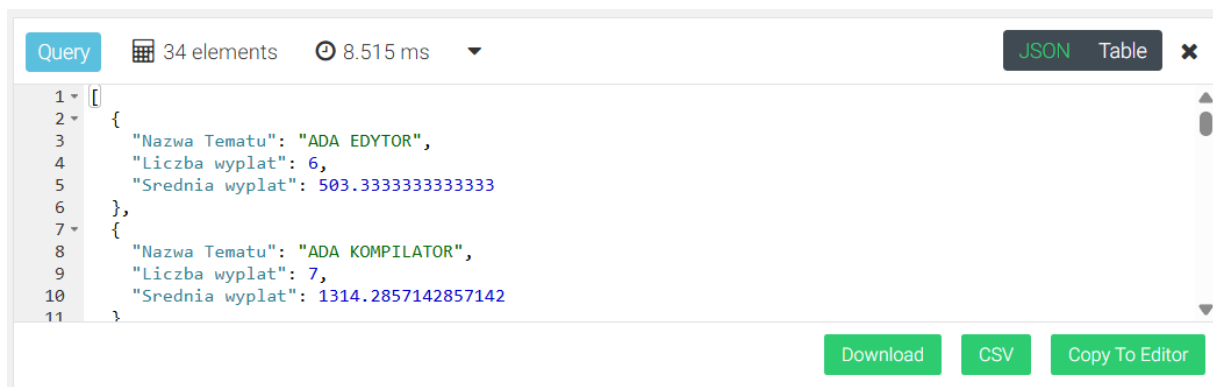
COLLECT temat = t.TEMAT

AGGREGATE liczba = COUNT(w),

srednia = AVERAGE(w.KWOTA)

RETURN {"Nazwa Tematu": temat,"Liczba wypłat": liczba,

"Srednia wypłat": srednia }



```
1 [
2   {
3     "Nazwa Tematu": "ADA EDYTOR",
4     "Liczba wypłat": 6,
5     "Srednia wypłat": 503.3333333333333
6   },
7   {
8     "Nazwa Tematu": "ADA KOMPILATOR",
9     "Liczba wypłat": 7,
10    "Srednia wypłat": 1314.2857142857142
11  }
]
```

Download CSV Copy To Editor

12. Podać średnie wypłaty dla każdego pracownika (wyświetl nazwisko i nr pracownika)

FOR p IN pracownicy

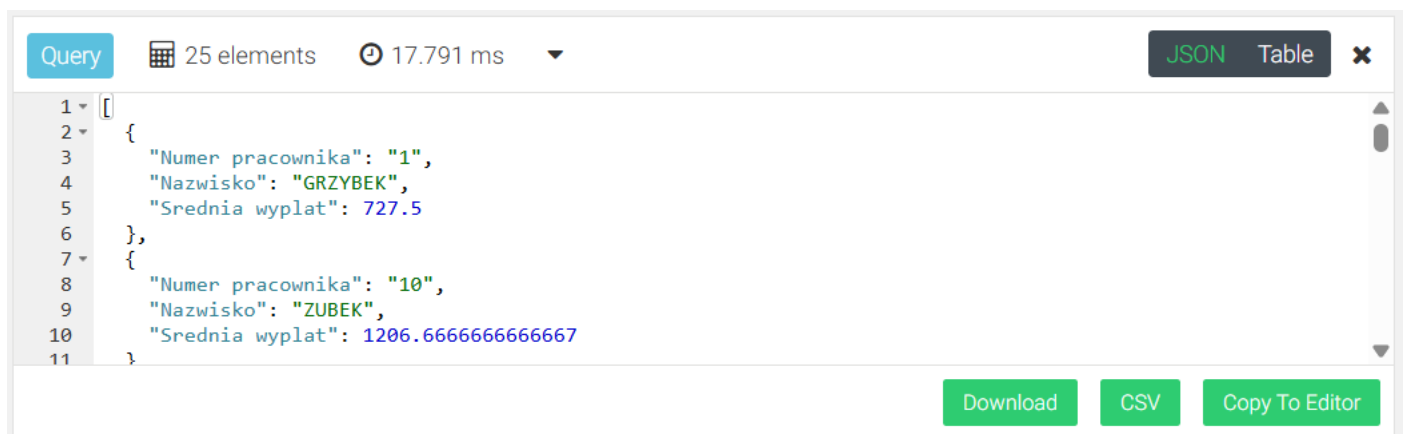
FOR w IN wypłaty

FILTER w.NR_TEM == p._key

COLLECT nr_prac = p._key, nazwisko = p.NAZWISKO

AGGREGATE srednia = AVERAGE(w.KWOTA)

RETURN {"Numer pracownika": nr_prac,"Nazwisko": nazwisko, "Srednia wypłat": srednia }



```
1 [
2   {
3     "Numer pracownika": "1",
4     "Nazwisko": "GRZYBEK",
5     "Srednia wypłat": 727.5
6   },
7   {
8     "Numer pracownika": "10",
9     "Nazwisko": "ZUBEK",
10    "Srednia wypłat": 1206.6666666666667
11  }
]
```

Download CSV Copy To Editor

13. Podać daty urodzenia najstarszych pracowników przydzielonych do każdej z funkcji

FOR p IN pracownicy

FOR prz IN przydzialy

FILTER prz.NR_PRAC == p._key

COLLECT funkcja = prz.KOD_FUNKCJI

AGGREGATE minimus = MIN(p.DATA_UR)

RETURN {"Kod funkcji": funkcja,"Max wiek": minimus }

Query 6 elements 16.155 ms

JSON Table

```
1 {
2   {
3     "Kod funkcji": "ADM",
4     "Max wiek": "1934-12-25 00:00:00"
5   },
6   {
7     "Kod funkcji": "ANL",
8     "Max wiek": "1934-10-05 00:00:00"
9   },
10  {
11    "Kod funkcji": "KRW"
```

Download CSV Copy To Editor

14. Podać wszystkie tematy, w których są przydziały do funkcji Tester i liczbę tych przydziałów

FOR f IN funkcje

FILTER f.NAZWA_FUNKCJI == "Tester"

FOR prz IN przydzialy

FILTER prz.KOD_FUNKCJI == f._key

COLLECT temat = prz.NR_TEM

AGGREGATE liczba = COUNT(1)

RETURN {"Numer tematu": temat,"Liczba przydziałów": liczba}

Query 14 elements 1.101 ms

JSON Table

```
1 {
2   {
3     "Numer tematu": "104",
4     "Liczba przydziałów": 2
5   },
6   {
7     "Numer tematu": "105",
8     "Liczba przydziałów": 1
9   },
10  {
11    "Numer tematu": "106"
```

Download CSV Copy To Editor

15. Podać wszystkie tematy, dla których suma wypłat jest mniejsza od 400.

```

FOR t IN tematy

  FOR w IN wypłaty

    FILTER w.NR_TEM == t._key

    COLLECT temat = t.TEMAT

      AGGREGATE suma = SUM(w.KWOTA)

      FILTER suma < 400

RETURN {"Nazwa Tematu": temat,"Suma": suma}

```

Query 1 elements 15.448 ms

JSON Table

```

1 [
2   {
3     "Nazwa Tematu": "SLABOPLATNY",
4     "Suma": 90
5   }
6 ]

```

Download CSV Copy To Editor

16. Podać nazwiska wszystkich pracowników, którzy są przydzieleni do więcej niż 5 różnych tematów

```

FOR p IN pracownicy

  FOR prz IN przydziały

    FILTER prz.NR_PRAC == p._key

    COLLECT pracownik = p._key

      AGGREGATE total = COUNT(1)

      FILTER total > 5

RETURN {"Numer pracownika": pracownik,"Ilość": total}

```

Query 25 elements 10.204 ms

JSON Table

```

1 [
2   {
3     "Numer pracownika": "1",
4     "Ilość": 20
5   },
6   {
7     "Numer pracownika": "11",
8     "Ilość": 8
9   },
10  {
11   "Numer pracownika": "12",

```

Download CSV Copy To Editor

17. Podać tematy, których data rozpoczęcia jest wcześniejsza niż data rozpoczęcia tematu DYSK

```
LET dyskData = (  
  FOR t in tematy  
  FILTER t.TEMAT == "DYSK"  
  LIMIT 1  
  RETURN t.DATA_ROZP  
)  
[0]  
FOR t IN tematy  
FILTER t.DATA_ROZP < dyskData  
RETURN {"TEMAT": t.TEMAT, "Data": t.DATA_ROZP}
```

Query 12 elements 1.155 ms JSON Table ✕

```
1 [
2 {
3   "TEMAT": "PASCAL KOMPILATOR",
4   "Data": "1983-03-21 00:00:00"
5 },
6 {
7   "TEMAT": "PROJEKTOWANIE UKLADOW",
8   "Data": "1983-12-11 00:00:00"
9 },
10 {
11   "TEMAT": "LISP - KOMPILATOR"
```

Download CSV Copy To Editor

18. Podać nazwiska pracowników, których suma wypłat jest większa niż suma wypłat pracownika MISIURY

```
LET SumaMisiura = (  
  FOR p in pracownicy  
  FILTER p.NAZWISKO == "MISIURA"  
  
  FOR w in wypłaty  
  FILTER w.NR_PRAC == p._key  
  
  COLLECT  
  
  AGGREGATE suma = SUM(w.KWOTA)  
  
  RETURN suma  
)
```

```
FOR p IN pracownicy  
  FOR w in wypłaty  
  FILTER w.NR_PRAC == p._key  
  
  COLLECT nazwisko = p.NAZWISKO  
  
  AGGREGATE suma = SUM(w.KWOTA)  
  
  FILTER suma > SumaMisiura  
  
RETURN {"Pracownik": nazwisko, "Suma": suma}
```

Query 28 elements 15.037 ms JSON Table ×

```
1  [
2  {
3    "Pracownik": "BIERNAT",
4    "Suma": 4190
5  },
6  {
7    "Pracownik": "FULAK",
8    "Suma": 4770
9  },
10 {
11  "Pracownik": "GRZYREK"
```

Download CSV Copy To Editor

19a. Podać tematy, do których jest więcej przydziałów niż do tematu DYSK

```
LET PrzydzialyDysk = (  
  FOR t in tematy  
  FILTER t.TEMAT == "DYSK"  
  
  FOR prz in przydzialy  
    FILTER prz.NR_TEM == t._key  
  
    COLLECT  
  
    AGGREGATE total = COUNT(1)  
  
  RETURN total  
)[0]  
  
FOR t in tematy  
  FOR prz in przydzialy  
    FILTER prz.NR_TEM == t._key  
  
    COLLECT temat = t.TEMAT  
  
    AGGREGATE total = COUNT(1)  
  
    FILTER total > PrzydzialyDysk  
  
  RETURN {"Temat": temat, "Ilosc": total}
```

Query 5 elements 14.825 ms JSON Table ×

```
1  [
2  {
3    "Temat": "KANAL WIZJI - OPROGRAMOWANIE",
4    "Ilosc": 11
5  },
6  {
7    "Temat": "KARTA EMULATORA PC",
8    "Ilosc": 10
9  },
10 {
11  "Temat": "MODUL I/O"
```

Download CSV Copy To Editor

19b. Podać tematy, do których przydzielono więcej pracowników niż do tematu DYSK

```
LET PrzydzialyDysk = LENGTH(  
  FOR t in tematy  
  FILTER t.TEMAT == "DYSK"  
  FOR prz in przydzialy  
    FILTER prz.NR_TEM == t._key  
  RETURN DISTINCT prz.NR_PRAC  
)
```

RETURN PrzydzialyDysk

```
FOR t in tematy  
  FOR prz in przydzialy  
    FILTER prz.NR_TEM == t._key  
  COLLECT temat = t.TEMAT  
  AGGREGATE total = COUNT_DISTINCT(prz.NR_PRAC)  
  FILTER total > PrzydzialyDysk  
RETURN {"Temat": temat, "Ilosc": total}
```

Query	20 elements	20.756 ms	JSON	Table	x
1	[				
2	{				
3	"Temat": "ADA EDYTOR",				
4	"Ilosc": 7				
5	},				
6	{				
7	"Temat": "ADA KOMPIlator",				
8	"Ilosc": 7				
9	},				
10	{				
11	"Temat": "BAZA DANYCH - PZU",				
12	"Ilosc": 7				
13	},				